Jetson Orin Nano에서 TensorRT로 최적화된 YOLOv11 객체 탐지 모델의 성능 분석

한윤욱[1]*, 김상현[1]
청주대학교 기계항공시스템공학과[1]

Performance Analysis of YOLOv11 Object Detection Model Optimized with TensorRT on Jetson Orin Nano

Yunuk Han[1]*, Sang-Hyeon Kim[1]

Key Words : Deep Learning(딥러닝), Object Detection(객체 탐지), DNN Model Optimization(심층 신경망 모델 최적화), Jetson Orin Nano(젯슨 오린 나노)

서 론

최근 무인항공기(UAV, Unmanned Aerial Vehicle)의 자율 임무 수행 능력이 발전함에 따라 온보드 비전 시스템을 기반으로 한 실시간 상황 인식 기술이 핵심 요소로 부상하고 있다. 충돌 회피, 지상 표적 탐지, 자동 착륙과 같은 복합 임무는 외부 인프라의 지원 없이 기체 자체의 판단에 의존해야 하므로 경량화된 인공지능 기반 영상 인식 기술의 중요성이 더욱 커지고 있다. 그러나 UAV 플랫폼은 크기(Size), 무게(Weight), 전력(Power)의 제약(SWaP constraints)으로 인해 고성능 AI 연산을 위한 임무 컴퓨터 탑재에 기술적 어려움이 따른다. 이러한 한계로 인해 실제 임베디드 환경에서 효율적인 추론 성능을 확보하는 것은 UAV 자율비행 기술의 핵심 과제로 제기된다.

이에 본 연구에서는 이를 검증하기 위해 엣지 AI 컴퓨터인 NVIDIA Jetson Orin Nano를 임무 컴퓨터로 선정하고 공개된 최적화 파이프라인(PyTorch → ONNX → TensorRT)를 사용하였다. YOLOv11 객체 탐지 모델을 대상으로 PyTorch 모델과 TensorRT 변환 모델의 End-to-End 추론 속도, GPU 활용률, 전력 소비량(mW 단위)을 정량적으로 비교 분석하였다.

본 론

YOLOv11 객체 탐지와 TensorRT 변환

본 연구에서는 실시간 객체 탐지를 위해 널리 사용되는 YOLO 아키텍처 기반의 YOLOv11 모델을 사용하였다. UAV와 같은 엣지 환경에서 심층 신경망 모델을 효율적으로 구동하기 위해서는 하드웨어에 특화된 최적화 과정이 필수적이다. 이를 위해 본 연구에서는 Ultralytics 프레임워크에서 제공하는 내장 export 기능을 활용하여 PyTorch 모델을 최적화하였다. 이 기능은 PyTorch 모델을 입력받아 내부적으로 다양한 프레임워크 간 호환성을 보장하는 개방형 표준인 ONNX(Open Neural Network Exchange)로 변환하는 과정을 거친다. 변환된 ONNX 모델을 NVIDIA의 추론 최적화 라이브러리인 TensorRT를 이용해 최종적으로 FP16(16비트 부동소수점) 정밀도를 갖는 엔진으로 변환하는 자동화된 파이프라인을 갖는다. 이 과정에서 TensorRT는 레이어 융합(Layer Fusion), 정밀도 축소(Precision Calibration), 커널 자동 튜닝(Kernel Auto-Tuning) 등의 최적화 기법을 적용하여 Jetson 하드웨어에 적합한 연산 구조를 구성한다.

하드웨어 및 소프트웨어 실험 환경

본 연구에서는 TensorRT 최적화에 따른 성능 변화를 정량적으로 측정하고자 Table 1의 환경에서 실험하였다. 실험 플랫폼으로는 NVIDIA Ampere 아키텍처 GPU가 내장된 Jetson Orin Nano Developer Kit을 25W 모드로 사용하였으며 시스템은 JetPack 6.2.1을 기반으로 CUDA(v12.9), TensorRT(v10.3) 등의 라이브러리를 활용했다. 영상 입력은 Intel Realsense D435i 카메라로 실시간 획득하고 GStreamer 파이프라인으로 CPU 부하를 최소화하며 데이터 스트림의 안정성을 확보했다. 성능 비교 대상은 COCO dataset으로 학습된 Ultralytics의 YOLOv11n.pt(PyTorch 기반) 모델과 이것을 FP16 정밀도로 최적화한 TensorRT 엔진(yolov11n.engine)이며 실시간 영상 속 단일 객체인 여객기 모델을 탐지 대상으로 하여 실험을 수행하였다.

Table 1. Experiment Setup

Hardware	
Device	Jetson Orin Nano Developer Kit(25W Mode)
CPU	6-core Arm® Cortex®-A78AE v8.2 64-bit (1.5MB L2 + 4MB L3)
GPU	1,024-core NVIDIA Ampere architecture with 32 Tensor Cores
RAM	8 GB 128-bit LPDDR5
Camera	Intel RealSense D435i (640x480)
Software	
System	JetPack 6.2.1 (L4T 36.4.4)
Framework	PyTorch 2.5.0, Ultralytics 8.3.195
Libraries	CUDA 12.9.86, TensorRT 10.3.0.30 OpenCV 4.10.0 with CUDA SDK 2.57.2 with Gstreamer 1.20.3

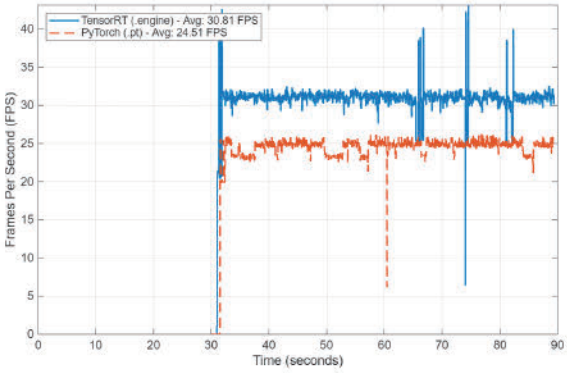

Fig. 1. End-to-End FPS comparison between PyTorch and TensorRT models

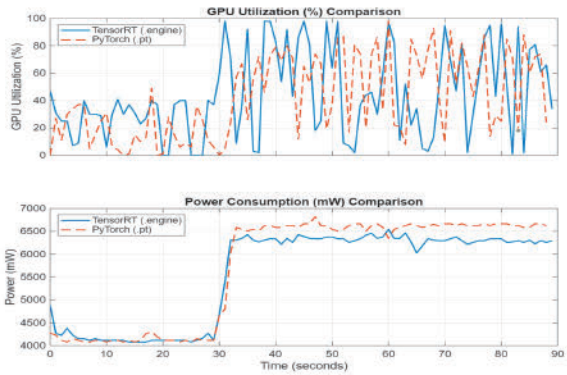

Fig. 2. GPU and Power Consumption Comparison

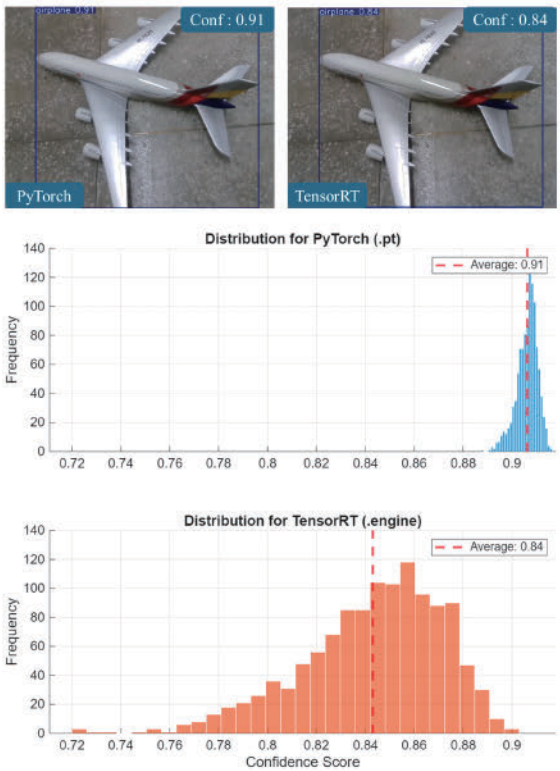

Fig. 3. Confidence Score Distribution of Models

실험 결과

본 연구에서 수행한 모든 실험은 30초의 워밍업 이후 60초간의 측정 구간 동안 진행되었으며 End-to-End FPS, GPU 활용률, 전력 소비량, 그리고 Confidence Score 분포를 분석하였다. 여기서 End-to-End FPS란 입력 영상 획득부터 모델 추론 및 결과 출력까지 전체 파이프라인을 포함한 실제 처리 속도를 의미한다.

PyTorch 원본 모델과 TensorRT 최적화 모델의 시간에 따른 End-to-End 추론 FPS 변화를 비교한 결과는 Fig. 1과 같다. 측정 결과 TensorRT 최적화를 적용한 모델은 평균 30.81 FPS를 기록하여 PyTorch 모델의 24.51 FPS 대비 약 25.7% 성능이 향상되었다.

Fig. 2는 GPU 활용률과 전력 소비량의 변화를 보여준다. 평균 GPU 활용률은 PyTorch 모델이 41.91%, TensorRT 모델이 44.06%로 측정되어 최적화 이후 GPU가 더 적극적으로 활용되었음을 확인하였다. 전력 소비량 그래프에서는 TensorRT 모델이 더 높은 GPU 활용률을 보였음에도 불구하고 평균 전력 소비량이 5567.09mW로 PyTorch 모델의 5728.82mW 대비 약 2.82% 감소하였다. 이는 FP16 변환을 통해 GPU의 Tensor Cores가 효율적으로 연산하며 와트당 성능비가 개선된 결과로 해석된다.

각각의 두 모델이 탐지한 객체의 Confidence Score 분포를 비교한 결과는 Fig. 3과 같다. PyTorch 모델은 평균 0.91로 대부분의 탐지에서 좁은 범위의 높은 신뢰도를 보였으나 TensorRT 모델은 평균 0.84로 더 넓은 범위의 분포를 나타냈다. 이는 FP16 변환 과정에서 발생한 미세한 정밀도 손실이 반영된 결과로 판단된다.

결 론

본 논문은 Jetson Orin Nano에서 YOLOv11 모델을 TensorRT로 최적화했을 때의 성능 변화를 분석하였다. 실험 결과 TensorRT 최적화는 소폭의 정확도 손실만으로 FPS와 전력 효율을 크게 향상시킴을 확인하였다.

후 기

이 논문은 2025~2027년도 청주대학교 연구장학 지원에 의한 것임.

참고문헌

1) Sang-Hyeon Kim and Yunuk Han, "Inference Speed Comparison of ResNet-Based Object Classifier Using PyTorch, ONNX, and TensorRT," *Proceedings of the 2025 KSAS Spring Conference*, 2025, pp. 1370-1371.

2) Rey, L., et al., "A Performance Analysis of You Only Look Once Models for Deployment on Constrained Computational Edge Devices in Drone Applications," *Electronics*, Vol. 1, No. 0, 2025, pp. 1-25.